

WEEK-8.1

2303A51804

Batch-28

Task-1:Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.
- Requirements:
 - o Password must have at least 8 characters.
 - o Must include uppercase, lowercase, digit, and special characters
 - o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1:

- Password validation logic passing all AI-generated test cases.

Prompt:

```
'''
#generate atleast 5 general test cases for is_strong_password and implement the validator function it should require password
# must have at least 8 characters amd it should contain uppercase,lowercase,digit and special character it should not contain spaces
'''
```

General testcases:

```
import re
def is_strong_password(password):
    if len(password) < 8:
        return False
    if not re.search(r'[A-Z]', password):
        return False
    if not re.search(r'[a-z]', password):
        return False
    if not re.search(r'[0-9]', password):
        return False
    if not re.search(r'[@$!%*?&]', password):
        return False
    if re.search(r'\s', password):
        return False
    return True
# Test cases
test_cases = [
    ("Password123!", True),
    ("weakpass", False),
    ("StrongPass1@", True),
    ("NoSpecialChar1", False),
    ("Short1!", False),
    ("ValidPass@2024", True),
    ("Space InPass1!", False)
]
for password, expected in test_cases:
    result = is_strong_password(password)
```

Output:

```
25     ("ValidPass@2024", True),
26     ("Space InPass1!", False)
27 ]
28 for password, expected in test_cases:
29     result = is_strong_password(password)
30     print(f"Password: {password}, Expected: {expected}, Result: {result}, Test Pa
31
32
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\exam.py'				
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c; cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\exam.py'				
Password: Password123!, Expected: True, Result: True, Test Passed: True				
Password: weakpass, Expected: False, Result: False, Test Passed: True				
Password: StrongPass1@, Expected: True, Result: True, Test Passed: True				
Password: NoSpecialChar1, Expected: False, Result: False, Test Passed: True				
Password: Short1!, Expected: False, Result: False, Test Passed: True				
Password: ValidPass@2024, Expected: True, Result: True, Test Passed: True				
Password: Space InPass1!, Expected: False, Result: False, Test Passed: True				

ASSERT TESTCASE:

Prompt:

```
| print(f'Password: {password}, Expected: {expected}, Result: {result}, Test Passed: {test_passed}')
#generate code for a atleast 5 assert test cases for is_strong_password
assert is_strong_password("Password123!") == True
```

Code and Output:

```
31 #generate code for a atleast 5 assert test cases for is_strong_password
32 assert is_strong_password("Password123!") == True
33 assert is_strong_password("weakpass") == False
34 assert is_strong_password("StrongPass1@") == True
35 assert is_strong_password("NoSpecialChar1") == False
36 assert is_strong_password("Short1!") == False
37 assert is_strong_password("ValidPass@2024") == True
38 assert is_strong_password("Space InPass1!") == False
39 print("All assert test cases passed!")
40
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> & 'c:\Users\manis\AppData\Local\Programs\Python\Python310\python.exe' -u -i n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher '52771' '--' 'C:\Users\manis\OneDrive\Desktop\PYTHON TRA'
Password: Password123!, Expected: True, Result: True, Test Passed: True
Password: weakpass, Expected: False, Result: False, Test Passed: True
Password: StrongPass1@, Expected: True, Result: True, Test Passed: True
Password: NoSpecialChar1, Expected: False, Result: False, Test Passed: True
Password: Short1!, Expected: False, Result: False, Test Passed: True
Password: ValidPass@2024, Expected: True, Result: True, Test Passed: True
Password: Space InPass1!, Expected: False, Result: False, Test Passed: True
All assert test cases passed!
○ PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA>

UNIT TESTCASES:

Prompt:

```
print(f'All assert test cases passed!')
#generate code for a atleast 5 unit test cases for is_strong password
```

CODE :

```
#####  
#generate code for a atleast 5 unit test cases for is_strong password  
import unittest  
class TestPasswordValidator(unittest.TestCase):  
    def test_valid_password(self):  
        self.assertTrue(is_strong_password("Password123!"))  
        self.assertTrue(is_strong_password("StrongPass1@"))  
        self.assertTrue(is_strong_password("ValidPass@2024"))  
    def test_invalid_password(self):  
        self.assertFalse(is_strong_password("weakpass"))  
        self.assertFalse(is_strong_password("NoSpecialChar1"))  
        self.assertFalse(is_strong_password("Short1!"))  
        self.assertFalse(is_strong_password("Space InPass1!"))  
if __name__ == '__main__':  
    unittest.main()
```

OUTPUT:

..

Ran 2 tests in 0.001s

OK

D:\C:\Users\manic\OneDrive\Desktop\PYTHON TRAI

PYTEST CASE:

Prompt:

```
| unittest.main()  
#generate code for atleast 5 pytest case for is_strong_password
```

CODE AND OUTPUT:

```
54 #generate code for atleast 5 pytest case for is_strong_password
55 import pytest
56 @pytest.mark.parametrize("password, expected", [
57     ("Password123!", True),
58     ("weakpass", False),
59     ("StrongPass1@", True),
60     ("NoSpecialChar1", False),
61     ("Short1!", False),
62     ("ValidPass@2024", True),
63     ("Space InPass1!", False)
64 ])
65 def test_is_strong_password(password, expected):
66     assert is_strong_password(password) == expected
67 if __name__ == "__main__":
68     pytest.main()
69
70
71
72
```

ROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
S C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c::; cd 'c:\Users\manis\OneDrive\Desktop\
hon.exe' 'c:\Users\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundl
THON TRA\exam.py'
```

K

```
S C:\Users\manis\OneDrive\Desktop\PYTHON TRA> python -m pytest exam.py
===== test session start
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\manis\OneDrive\Desktop\PYTHON TRA
plugins: anyio-4.8.0
collected 9 items

exam.py .....

```

Explanation:

The code above implements a password strength validator function called `'is_strong_password'`. This function checks if a given password meets certain criteria to be considered strong. The criteria include:

1. The password must be at least 8 characters long.
2. It must contain at least one uppercase letter.
3. It must contain at least one lowercase letter.
4. It must contain at least one digit.

5. It must contain at least one special character from the set `@\$!%\*?&`.

6. It must not contain any spaces.

The function uses regular expressions to check for the presence of uppercase letters, lowercase letters, digits, special characters, and spaces. If any of the criteria are not met, the function returns `False`. If all criteria are satisfied, it returns `True`.

TASK-2:

Task Description #2 (Number Classification with Loops Apply AI for Edge Case Handling)

General Testcase:

Prompt:

```
#Number classification with loops use AI to generate atleast 5 general test cases for classify_number(n) function
```

Code:

```
def classify_number(n):
    if n < 0:
        return "Negative"
    elif n == 0:
        return "Zero"
    else:
        return "Positive"

user_number = int(input("Enter a number: "))
result = classify_number(user_number)
print("Result:", result)

test_cases = [
    (-5, "Negative"),
    (0, "Zero"),
    (10, "Positive"),
    (-1, "Negative"),
    (100, "Positive")
]

print("\nRunning test cases...\n")
for test_input, expected in test_cases:
    result = classify_number(test_input)
    print(f"Input: {test_input}, Expected: {expected}, Result: {result}, Test Passed: {result == expected}")
```

Output:

```
Enter a number: -5
Result: Negative
```

Running test cases...

```
Input: -5, Expected: Negative, Result: Negative, Test Passed: True
Input: 0, Expected: Zero, Result: Zero, Test Passed: True
Input: 10, Expected: Positive, Result: Positive, Test Passed: True
Input: -1, Expected: Negative, Result: Negative, Test Passed: True
Input: 100, Expected: Positive, Result: Positive, Test Passed: True
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA>
```

ASSERT TESTCASE:

Prompt:

```
#generate code for a atleast 5 assert test cases for classify_number(n) function
```

CODE AND OUTPUT:

```

104 assert classify_number(-5) == "Negative"
105 assert classify_number(0) == "Zero"
106 assert classify_number(10) == "Positive"
107 assert classify_number(-1) == "Negative"
108 assert classify_number(100) == "Positive"
109 print("All assert test cases passed!")
110
111
112
113
114

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> & 'c:\Users\manis\AppData\Local\Programs\Python\Py  
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '65063' '--' 'C:\Users\manis\OneDrive  
Enter a number: 0  
Result: Zero
```

Running test cases...

```
Input: -5, Expected: Negative, Result: Negative, Test Passed: True
Input: 0, Expected: Zero, Result: Zero, Test Passed: True
Input: 10, Expected: Positive, Result: Positive, Test Passed: True
Input: -1, Expected: Negative, Result: Negative, Test Passed: True
Input: 100, Expected: Positive, Result: Positive, Test Passed: True
All assert test cases passed!
```

UNIT TESTCASE:

Prompt:

```
print('All assert test cases passed.')  
#generate code for a atleast 5 unit test cases for classify_number(n) function
```

CODE AND OUTPUT:

```
111 import unittest  
112 class TestClassifyNumber(unittest.TestCase):  
113     def test_negative_number(self):  
114         self.assertEqual(classify_number(-5), "Negative")  
115         self.assertEqual(classify_number(-1), "Negative")  
116     def test_zero(self):  
117         self.assertEqual(classify_number(0), "Zero")  
118     def test_positive_number(self):  
119         self.assertEqual(classify_number(10), "Positive")  
120         self.assertEqual(classify_number(100), "Positive")  
121 if __name__ == '__main__':  
122     unittest.main()  
123  
124  
125
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:; cd 'c:\Users\manis\OneDrive\Desktop\PYTHON  
thon.exe' 'c:\Users\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs  
YTHON TRA\exam.py'
```

```
Input: -5, Expected: Negative, Result: Negative, Test Passed: True  
Input: 0, Expected: Zero, Result: Zero, Test Passed: True  
Input: 10, Expected: Positive, Result: Positive, Test Passed: True  
Input: -1, Expected: Negative, Result: Negative, Test Passed: True  
Input: 100, Expected: Positive, Result: Positive, Test Passed: True  
All assert test cases passed!
```

```
...
```

```
-----  
Ran 3 tests in 0.000s
```

PYTEST CASE:

Prompt:

```
unittest.main()  
#generate code for atleast 5 pytest case for classify_number(n) function  
import pytest
```


CODE AND OUTPUT:

```
124 import pytest
125 @pytest.mark.parametrize("input, expected", [
126     (-5, "Negative"),
127     (0, "Zero"),
128     (10, "Positive"),
129     (-1, "Negative"),
130     (100, "Positive")
131 ])
132 def test_classify_number(input, expected):
133     assert classify_number(input) == expected
134 if __name__ == "__main__":
135     pytest.main()
136
137
138
139
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Users\manis\OneDrive\Desktop\PYTHON TRA> python -m exam.py
Enter a number: 0
Result: Zero

Running test cases...

Input: -5, Expected: Negative, Result: Negative, Test Passed: True
Input: 0, Expected: Zero, Result: Zero, Test Passed: True
Input: 10, Expected: Positive, Result: Positive, Test Passed: True
Input: -1, Expected: Negative, Result: Negative, Test Passed: True
Input: 100, Expected: Positive, Result: Positive, Test Passed: True
All assert test cases passed!
```

Explanation:

Classify _number(n)` that classifies a given number `n` as "Negative", "Zero", or "Positive" based on its value.

1. If `n` is less than 0, it returns "Negative".
2. If `n` is equal to 0, it returns "Zero".
3. If `n` is greater than 0, it returns "Positive".

The code also includes a section for testing the function with various test cases. It uses assert statements to verify that the function returns the expected results for different inputs.

TASK-3:

(Anagram Checker – Apply AI for String Analysis)

Prompt:

```
# generate at least 5 general test cases for is_anagram(str1,str2) and implement the function  
def is_anagram(str1, str2):
```

Code:

```
def is_anagram(str1, str2):  
    str1 = str1.replace(" ", "").lower()  
    str2 = str2.replace(" ", "").lower()  
    return sorted(str1) == sorted(str2)  
  
# Test cases  
test_cases = [  
    ("listen", "silent", True),  
    ("triangle", "integral", True),  
    ("apple", "pabble", False),  
    ("Dormitory", "Dirty Room", True),  
    ("Conversation", "Voices Rant On", True),  
    ("Hello", "World", False)  
]  
  
for str1, str2, expected in test_cases:  
    result = is_anagram(str1, str2)  
    print(f"String 1: '{str1}', String 2: '{str2}', Expected: {expected}, Result: {result}, Test Passed: {result == expected}")
```

Output:

```
ms-python-debugpy-2025.18.0-win32-x64\bin\debugpy_launcher -- C:\Users\manis\OneDrive\Desktop\Python TRA  
String 1: 'listen', String 2: 'silent', Expected: True, Result: True, Test Passed: True  
String 1: 'triangle', String 2: 'integral', Expected: True, Result: True, Test Passed: True  
String 1: 'apple', String 2: 'pabble', Expected: False, Result: False, Test Passed: True  
String 1: 'Dormitory', String 2: 'Dirty Room', Expected: True, Result: True, Test Passed: True  
String 1: 'Conversation', String 2: 'Voices Rant On', Expected: True, Result: True, Test Passed: True  
String 1: 'Hello', String 2: 'World', Expected: False, Result: False, Test Passed: True  
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA>
```

Assert Testcase:

Prompt:

```
print(f"String 1: {str1}, String 2: {str2}, Expected: {expected}, Result: {result}")  
#generate code for a atleast 5 assert test cases for is_anagram(str1,str2) function  
assert is_anagram("listen", "silent") == True
```

Code:

```
#generate code for a atleast 5 assert test cases for is_anagram(str1,str2)
assert is_anagram("listen", "silent") == True
assert is_anagram("triangle", "integral") == True
assert is_anagram("apple", "pabble") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("Conversation", "Voices Rant On") == True
assert is_anagram("Hello", "World") == False
print("All assert test cases passed!")
```

Output:

```
String 1: 'listen', String 2: 'silent', Expected: True, Result: True, Test Passed: True
String 1: 'triangle', String 2: 'integral', Expected: True, Result: True, Test Passed: True
String 1: 'apple', String 2: 'pabble', Expected: False, Result: False, Test Passed: True
String 1: 'Dormitory', String 2: 'Dirty Room', Expected: True, Result: True, Test Passed: True
String 1: 'Conversation', String 2: 'Voices Rant On', Expected: True, Result: True, Test Passed: True
String 1: 'Hello', String 2: 'World', Expected: False, Result: False, Test Passed: True
All assert test cases passed!
D:\C++\Projects\manic\OneDrive\Desktop\Python\Tuples
```

Unit Testcase:

Prompt:

```
#generate code for a atleast 5 unit test cases for is_anagram(str1,str2) function
```

Code:

```
import unittest
class TestAnagram(unittest.TestCase):
    def test_anagram(self):
        self.assertTrue(is_anagram("listen", "silent"))
        self.assertTrue(is_anagram("triangle", "integral"))
        self.assertFalse(is_anagram("apple", "pabble"))
        self.assertTrue(is_anagram("Dormitory", "Dirty Room"))
        self.assertTrue(is_anagram("Conversation", "Voices Rant On"))
        self.assertFalse(is_anagram("Hello", "World"))
if __name__ == '__main__':
    unittest.main()
```

Output:

```
String 1: 'listen', String 2: 'silent', Expected: True, Result: True, Test Passed: True
String 1: 'triangle', String 2: 'integral', Expected: True, Result: True, Test Passed: True
String 1: 'apple', String 2: 'pabble', Expected: False, Result: False, Test Passed: True
String 1: 'Dormitory', String 2: 'Dirty Room', Expected: True, Result: True, Test Passed: True
String 1: 'Conversation', String 2: 'Voices Rant On', Expected: True, Result: True, Test Passed: True
String 1: 'Hello', String 2: 'World', Expected: False, Result: False, Test Passed: True
All assert test cases passed!
```

```
Ran 1 test in 0.001s
```

OK

Py test case:

Prompt:

```
#generate code for atleast 5 pytest case for is_anagram(str1,str2) function
```

Code:

```
import pytest
@pytest.mark.parametrize("str1, str2, expected", [
    ("listen", "silent", True),
    ("triangle", "integral", True),
    ("apple", "pabble", False),
    ("Dormitory", "Dirty Room", True),
    ("Conversation", "Voices Rant On", True),
    ("Hello", "World", False)
])
def test_is_anagram(str1, str2, expected):
    assert is_anagram(str1, str2) == expected
if __name__ == "__main__":
    pytest.main()
```

Output:

```
String 1: 'listen', String 2: 'silent', Expected: True, Result: True, Test Passed: True
OK
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> python -m pytest exam.py
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\manis\OneDrive\Desktop\PYTHON TRA
plugins: anyio-4.8.0
collected 7 items

exam.py .....
7 passed in 0.06s
```

Explanation:

1. `add_items(name, quantity)`: This method adds a specified quantity of an item to the inventory. If the item already exists, it updates the quantity; otherwise, it creates a new entry.
2. `remove_items(name, quantity)`: This method removes a specified quantity of an item from the inventory. It checks if the item exists and if there is enough stock to remove. If the removal is successful, it returns `True`; otherwise, it returns `False`.
3. `get_stock(name)`: This method returns the current stock quantity of a specified item. If the item does not exist, it returns `0`.
4. The code also includes test cases to validate the functionality of the `Inventory` class. It tests adding items, removing items, and retrieving stock levels.
5. The test cases are implemented using both assert statements and unit testing with the `unittest` framework, as well as using `pytest` for parameterized testing. Each test case checks the expected outcome against the actual result to ensure that the methods in the `Inventory` class work correctly.

Task-4: (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Prompt:

```
# generate at least 5 general test cases for an inventory class with stock management Methods an
```

Code:

```
195 class Inventory:
196     def __init__(self):
197         self.stock = {}
198     def add_items(self, name, quantity):
199         if name in self.stock:
200             self.stock[name] += quantity
201         else:
202             self.stock[name] = quantity
203     def remove_items(self, name, quantity):
204         if name in self.stock and self.stock[name] >= quantity:
205             self.stock[name] -= quantity
206             return True
207         return False
208     def get_stock(self, name):
209         return self.stock.get(name, 0)
210 # Test cases
211 inventory = Inventory()
212 inventory.add_items("apple", 50)
213 inventory.add_items("banana", 30)
214 print(inventory.get_stock("apple")) # Expected: 50
215 print(inventory.get_stock("banana")) # Expected: 30
216 inventory.remove_items("apple", 20)
217 print(inventory.get_stock("apple")) # Expected: 30
218 inventory.remove_items("banana", 40) # Expected: False
219 print(inventory.get_stock("banana")) # Expected: 30
220 inventory.add_items("orange", 20)
221 print(inventory.get_stock("orange")) # Expected: 20
222
```

Output:

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:: cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA'; & 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\debugpy_launcher' '56129' '--' '(
50
30
30
30
20
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> |
```

Assert testcase:

Prompt:

```
# generate code for a atleast 5 assert test cases for inventory class with stock management Methods and add_items(name,quality),remove_items(name,quantity)
```

Code:

```
5 inventory = Inventory()
5 inventory.add_items("apple", 50)
7 inventory.add_items("banana", 30)
3 assert inventory.get_stock("apple") == 50
3 assert inventory.get_stock("banana") == 30
3 inventory.remove_items("apple", 20)
1 assert inventory.get_stock("apple") == 30
2 assert inventory.remove_items("banana", 40) == False
3 assert inventory.get_stock("banana") == 30
4 inventory.add_items("orange", 20)
5 assert inventory.get_stock("orange") == 20
5 print("All assert test cases passed!")
7
3 |
```

Output:

```
50
30
30
30
20
All assert test cases passed!
```

Unit test case:

Prompt:

```
#generate code for atleast 5 unit test case for inventory class with stock management Methods and add_items(name,quality)
```

Code:

```
import unittest
class TestInventory(unittest.TestCase):
    def setUp(self):
        self.inventory = Inventory()
        self.inventory.add_items("apple", 50)
        self.inventory.add_items("banana", 30)
    def test_add_items(self):
        self.inventory.add_items("orange", 20)
        self.assertEqual(self.inventory.get_stock("orange"), 20)
    def test_remove_items(self):
        self.assertTrue(self.inventory.remove_items("apple", 20))
        self.assertEqual(self.inventory.get_stock("apple"), 30)
        self.assertFalse(self.inventory.remove_items("banana", 40))
        self.assertEqual(self.inventory.get_stock("banana"), 30)
    def test_get_stock(self):
        self.assertEqual(self.inventory.get_stock("apple"), 50)
        self.assertEqual(self.inventory.get_stock("banana"), 30)
        self.assertEqual(self.inventory.get_stock("orange"), 0)
if __name__ == '__main__':
    unittest.main()
```


Output:

```
xceptions \ms-python-debugpy-2023.10.0-win32-x64\bundled\libs\debugpy\la
50
30
30
30
20
All assert test cases passed!
...
-----
Ran 3 tests in 0.001s

OK
PS C:\Users\user> python test.py
Python 3.10.0 (tags/v3.10.0:2023-03-07) [AMD64] on win32
```

Pytest case:

Prompt:

```
generate code for atleast 5 pytest case for inventory class with stock management Methods and add_items(name,quality),rer
```

Code:

```
import pytest
@pytest.fixture
def inventory():
    inv = Inventory()
    inv.add_items("apple", 50)
    inv.add_items("banana", 30)
    return inv
def test_add_items(inventory):
    inventory.add_items("orange", 20)
    assert inventory.get_stock("orange") == 20
def test_remove_items(inventory):
    assert inventory.remove_items("apple", 20) == True
    assert inventory.get_stock("apple") == 30
    assert inventory.remove_items("banana", 40) == False
    assert inventory.get_stock("banana") == 30
def test_get_stock(inventory):
    assert inventory.get_stock("apple") == 50
    assert inventory.get_stock("banana") == 30
    assert inventory.get_stock("orange") == 0
if __name__ == "__main__":
    pytest.main()
```

Output:

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> python -m pytest exam.py
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\manis\OneDrive\Desktop\PYTHON TRA
plugins: anyio-4.8.0
collected 6 items

exam.py .....

===== 6 passed in 0.05s =====
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA>
```

Explanation:

- 1, The code defines an `Inventory` class that manages stock for different items.
2. The class has three methods: `add\_items(name, quantity)` to add stock, `remove\_items(name, quantity)` to remove stock, and `get\_stock(name)` to check the current stock of an item.

3. The ``add_items`` method updates the stock for an item, while the ``remove_items`` method checks if there is enough stock before removing and returns ``True`` or ``False`` accordingly.
4. The ``get_stock`` method returns the current stock for a given item, or 0 if the item is not in stock.
5. The code includes test cases using both ``assert`` statements and the ``unittest`` framework to validate the functionality of the ``Inventory`` class.

Task-5: Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates

General testcase:

Prompt:

```
#generate code for atleast 5 general test cases for validate_and_format_date(date_str) to check and convert dates in the format "DD/MM/YYYY" to "YYYY-MM-DD"
# import re
```

Code:

```
import re
def validate_and_format_date(date_str):
    pattern = r'^(0[1-9]|1[12][0-9]|3[01])/(0[1-9]|1[0-2])/d{4}$'
    if re.match(pattern, date_str):
        day, month, year = date_str.split('/')
        return f"{year}-{month}-{day}"
    else:
        return "Invalid date format"
# Test cases
test_cases = [
    ("15/08/2021", "2021-08-15"),
    ("01/01/2020", "2020-01-01"),
    ("31/12/1999", "1999-12-31"),
    ("30/02/2021", "Invalid date format"), # Invalid date
    ("12-10-2020", "Invalid date format"), # Incorrect format
    ("29/02/2020", "2020-02-29"), # Leap year
    ("31/04/2021", "Invalid date format") # April has 30 days
]
print("\nRunning test cases...\n")
for date_str, expected in test_cases:
    result = validate_and_format_date(date_str)
    print(f"Input: {date_str}, Expected: {expected}, Result: {result}, Test Passed: {result == expected}")
```

Output:

Running test cases...

```
Input: 15/08/2021, Expected: 2021-08-15, Result: 2021-08-15, Test Passed: True
Input: 01/01/2020, Expected: 2020-01-01, Result: 2020-01-01, Test Passed: True
Input: 31/12/1999, Expected: 1999-12-31, Result: 1999-12-31, Test Passed: True
Input: 30/02/2021, Expected: Invalid date format, Result: 2021-02-30, Test Passed: False
Input: 12-10-2020, Expected: Invalid date format, Result: Invalid date format, Test Passed: True
Input: 29/02/2020, Expected: 2020-02-29, Result: 2020-02-29, Test Passed: True
Input: 31/04/2021, Expected: Invalid date format, Result: 2021-04-31, Test Passed: False
```

Assert testcase:

Prompt:

```
# generate code for a atleast 5 assert test cases for validate_and_format_date(date_Str) function
```

Code:

```
assert validate_and_format_date("15/08/2021") == "2021-08-15"
assert validate_and_format_date("01/01/2020") == "2020-01-01"
assert validate_and_format_date("31/12/1999") == "1999-12-31"
assert validate_and_format_date("30/02/2021") == "Invalid date format"
assert validate_and_format_date("12-10-2020") == "Invalid date format"
assert validate_and_format_date("29/02/2020") == "2020-02-29"
assert validate_and_format_date("31/04/2021") == "Invalid date format"
print("All assert test cases passed!")
```

Output:

```
Input: 01/01/2020, Expected: 2020-01-01, Result: 2020-01-01, Test Passed: True
Input: 31/12/1999, Expected: 1999-12-31, Result: 1999-12-31, Test Passed: True
Input: 30/02/2021, Expected: Invalid date format, Result: 2021-02-30, Test Passed: False
Input: 12-10-2020, Expected: Invalid date format, Result: Invalid date format, Test Passed: True
Input: 29/02/2020, Expected: 2020-02-29, Result: 2020-02-29, Test Passed: True
Input: 31/04/2021, Expected: Invalid date format, Result: 2021-04-31, Test Passed: False
```

Unit test case:

Prompt:

```
# generate code for a atleast 5 unit test cases for validate_and_format_date(date_Str) function
```

Code:

```
import unittest
class TestValidateAndFormatDate(unittest.TestCase):
    def test_valid_dates(self):
        self.assertEqual(validate_and_format_date("15/08/2021"), "2021-08-15")
        self.assertEqual(validate_and_format_date("01/01/2020"), "2020-01-01")
        self.assertEqual(validate_and_format_date("31/12/1999"), "1999-12-31")
        self.assertEqual(validate_and_format_date("29/02/2020"), "2020-02-29") # Leap year
    def test_invalid_dates(self):
        self.assertEqual(validate_and_format_date("30/02/2021"), "Invalid date format") # Invalid date
        self.assertEqual(validate_and_format_date("12-10-2020"), "Invalid date format") # Incorrect format
        self.assertEqual(validate_and_format_date("31/04/2021"), "Invalid date format") # April has 30 days
if __name__ == '__main__':
    unittest.main()
```

Output:

```
Input: 15/08/2021, Expected: 2021-08-15, Result: 2021-08-15, Test Passed: True
Input: 30/02/2021, Expected: Invalid date format, Result: 2021-02-30, Test Passed: False
Input: 12-10-2020, Expected: Invalid date format, Result: Invalid date format, Test Passed: True
Input: 29/02/2020, Expected: 2020-02-29, Result: 2020-02-29, Test Passed: True
Input: 31/04/2021, Expected: Invalid date format, Result: 2021-04-31, Test Passed: False
All assert test cases passed!
```

Pytest case:

Prompt:

```
# generate code for atleast 5 pytest case for validate_and_format_date(date_Str) function
```

Code:

```
import pytest
@pytest.mark.parametrize("date_str, expected", [
    ("15/08/2021", "2021-08-15"),
    ("30/02/2021", "Invalid date format"), # Invalid date
    ("12-10-2020", "Invalid date format"), # Incorrect format
    ("29/02/2020", "2020-02-29"), # Leap year
    ("31/04/2021", "Invalid date format") # April has 30 days
])
def test_validate_and_format_date(date_str, expected):
    assert validate_and_format_date(date_str) == expected
if __name__ == '__main__':
    pytest.main()
```

Output:

```
----- test session starts -----
platform win32 -- Python 3.13.7, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\manis\OneDrive\Desktop\PYTHON TRA
plugins: anyio-4.8.0
collected 0 items

rootdir: C:\Users\manis\OneDrive\Desktop\PYTHON TRA
plugins: anyio-4.8.0
collected 0 items

===== no tests ran in 0.03s =====
```

Explanation:

1. The code defines a function ``validate_and_format_date(date_str)`` that takes a date string as input and checks if it is in the format "DD/MM/YYYY" using a regular expression.
2. The regular expression checks for valid day (01-31), month (01-12), and a four-digit year.
3. If the input date string matches the pattern, the function splits the string into day, month, and year components and returns the date in the format "YYYY-MM-DD".
4. If the input date string does not match the pattern, the function returns "Invalid date format".
5. The code includes test cases to validate the functionality of the ``validate_and_format_date`` function.